

System Architecture: Grid-Guardian AI-Blockchain-Edge Integration

Grid-Guardian tightly couples **agentic AI**, **blockchain smart contracts**, and **Raspberry Pi edge nodes** into a unified P2P energy trading system. Each component triggers the next in a predictable timeline. In brief:

- **Agentic AI (edge)** forecasts loads/production and decides buy/sell/storage actions on each Pi.
- **Blockchain (smart contracts)** automates trade matching, pricing, and settlement in a secure ledger.
- **Hardware (Raspberry Pi)** senses real power, runs AI inference, signs transactions, and actuates devices.

The timeline below walks step-by-step through a typical operation cycle, from sensing to trade execution, citing recent work on AI-driven decentralized trading ¹ ². Afterwards we recommend state-of-the-art technologies and protocols for each integration point.

1. Local Sensing & Forecasting (Edge AI triggers)

1. **Time-of-day trigger** (e.g. daily at midnight or upon significant weather update): each Raspberry Pi collects sensor data (meters, PV inverters, etc.) and external forecasts.
2. **Agentic AI inference** on the Pi runs predictive models (e.g. GRU/LSTM/Transformer) to **forecast next-day/hour household load and PV generation** ¹ ². These lightweight models (“EMS forecasting component”) are triggered by a daily timer ².
3. The **forecast results** feed into the local **Energy Management System (EMS)**. The EMS has two sub-modules: (a) a **Self-Controller (SC)** that decides on local demand-side actions (e.g. throttle appliances), and (b) a **Transaction Controller (TC)** that decides trading actions ² ³. Both SC and TC run on each Pi.
4. *Example:* If forecasts predict surplus PV and high prices, the Pi AI may label the node as a “seller” with an energy offer. If forecasts predict deficit, it becomes a “buyer” with a demand bid ¹.
5. The **hardware (sensors/IoT)** feeds back into the AI loop continuously: any deviation (e.g. sudden cloud cover) can trigger immediate re-evaluation of control actions by the AI on the Pi. This creates a closed-loop with real-time feedback.

2. Bid/Offer Generation (AI → Blockchain)

1. **Decision to trade:** Based on forecasts and current state (SoC, tariffs), the TC sub-module composes **energy sale or purchase offers**. Each offer includes quantity, price, and validity window.
2. **Digital commitment:** The Pi signs the offer (e.g. signs a hash of [timestamp | household_id | quantity | price]) and may optionally use a *commit-reveal scheme* to ensure honesty. This signing uses the Pi’s local blockchain key (securely stored).

3. **Submit to network:** The signed offer is broadcast to the P2P energy market. This can happen via a **blockchain transaction** or a side messaging network. In practice, the TC calls a Web3 library to send the offer data to a *Trading Smart Contract (TSC)* on the blockchain ³ ⁴ .
4. **Timing:** This happens on an hourly or sub-hourly cadence (configurable). In the cited design, TC publishes offers to the smart contract periodically (e.g. hourly) ³ . It could also respond to asynchronous events (like a sudden price spike).
5. **Fallback to offline matching:** For speed, the Pi could also exchange offers with local neighbors over a mesh (e.g. via MQTT or libp2p) before writing to chain. However, **final settlement always occurs on-chain** for security and auditability ⁵ ⁴ .

3. Blockchain Matching & Settlement

1. **Smart contract processing:** The **Trading Smart Contract (TSC)** collects all submitted buy/sell orders and runs a **market-clearing algorithm** (e.g. linear programming) to match supply and demand ⁵ . This happens on-chain (or via a trusted off-chain optimizer), typically on a daily or hourly schedule.
2. **Price discovery:** The smart contract computes transaction prices (e.g. uniform clearing price or negotiated price) and determines which offers succeed ⁵ . All this is done deterministically and transparently by the contract code.
3. **Commit–Reveal (if used):** If the system uses commit–reveal, participants reveal their salts at this stage so the contract can verify honest quantities. This enforces no double-spending of offers.
4. **Settlement:** The contract automatically settles trades by transferring funds (or credit) between accounts, escrowed from earlier deposits. All successful trades and prices are recorded in the blockchain ledger ⁶ .
5. **Example:** “Buyer1 purchases 5 kWh from Seller2 at price X. The blockchain debits Buyer1’s account, credits Seller2’s, and logs the trade.”
6. **Event emission:** Upon matching, the blockchain emits events (e.g. `TradeExecuted`) for each trade. Each Pi node’s TC listens for relevant events (filtered by its own ID).

4. Local Actuation (Blockchain → Hardware)

1. **Trade notification:** When a trade is confirmed on-chain, the Raspberry Pi **receives an event** via its blockchain client (e.g. via Web3 filters or a light node).
2. **Control action:** The Pi’s EMS acts on the trade. If the Pi sold energy, it will **discharge the battery or allow PV injection** into the grid as agreed. If it bought energy, it will **charge the battery or draw from the grid**. The SC sub-module translates the contract result into hardware commands (e.g. control the inverter or smart outlet) ² ³ .
3. **Safety shield:** Before executing, a **local safety controller** double-checks constraints (e.g. SoC limits, max current). If an action is unsafe, it overrides or clamps the command (e.g. never let SoC drop below minimum). This shield runs on the Pi and enforces hard safety rules independent of the contract logic.

5. Feedback, Learning, and Monitoring

1. **Performance telemetry:** The Pi logs actual production/consumption and realized trades back to a monitoring server or MLOps pipeline. This can also be anchored in the blockchain (e.g. post aggregated results on-chain for audit) ⁶.
2. **Retraining trigger:** At end-of-day or week, a central server collects logs from all nodes (or uses blockchain records) to retrain/update AI models (for forecasting, bidding strategy, etc.) via offline machine learning.
3. **Model rollout:** Updated models are pushed to edge nodes over secure updates (e.g. encrypted container images or binaries). Each Pi verifies signature before using the new model. This completes one lifecycle.

Integration Points & Protocols

- **Hardware↔AI (Edge):** Sensors (PZEM-004T meters, INA219, etc.) connect to each Pi via UART/RS485 or I²C. The Pi runs AI inference (PyTorch/TensorFlow Lite). Use **Docker** or **Python virtualenv** on Raspbian Linux.
- **AI↔Local Comm:** Edge AIs share status/offers over local network via **MQTT** (lightweight pub/sub) or **libp2p (gossipsub)**. This lets neighbors negotiate before or in parallel to blockchain settlement. Payloads are compact JSON or protobuf.
- **Edge↔Blockchain:** Each Pi runs a light Ethereum client (e.g. Geth in light mode) or connects to an L2 node via Web3. Communication uses **Web3 RPC** over HTTPS/TLS. Smart contracts are written in Solidity (or Cadence, etc.) and deployed on a scalable network (see Stack below). Each transaction is signed with the Pi's private key (stored in a hardware secure element or encrypted keystore).
- **Blockchain↔Network:** The smart contracts (TradingSC, IdentitySC, etc.) implement on-chain logic. Nodes connect via a **Permissioned Blockchain** (e.g. Hyperledger Besu, or a consortium PoS Ethereum network) for scalability. Agents can also use **oracles** (e.g. Chainlink) if external price or weather data is needed on-chain.
- **Security & Identity:** Use mTLS and PKI for Pi-server comms. Each Pi's blockchain identity is rooted in a secured key. Smart contracts manage identities via certificates and allow only registered nodes to trade ⁷. Use Diffie-Hellman or TLS to encrypt IoT telemetry.

Technology Stack Recommendations

- **Edge AI:** Python 3.10, PyTorch or TensorFlow. Use **Stable-Baselines3** or **HuggingFace Transformers** for policy models (including sequence models). Convert models to **TorchScript** or **ONNX** for fast inference on Pi. For reinforcement learning, frameworks like **RLlib** or **tianshou** can be used offline during training.
- **Hardware:** Raspberry Pi 5 (ARM64). OS: Raspberry Pi OS or Ubuntu. Use Docker or Podman containers for deployment isolation. Sensors: PZEM-004T (RS485) or INA219 for power/voltage. Include a real-time clock and UPS for reliability.
- **Networking:** Use **MQTT (Eclipse Mosquitto)** broker on local LAN for pub/sub of telemetry and offers (ports 1883/TLS). Alternatively, **gRPC** or **REST+JSON** APIs for inter-Pi calls. Ensure IPv6 or local mesh networking for P2P communication. Use **TLS 1.3** for all communications.
- **Blockchain:** An **Ethereum-compatible permissioned ledger** (e.g. Hyperledger Besu, or Consortium ETH on Polygon PoS). This ensures high TPS and finality. Smart contracts in Solidity for trading logic.

Use **Web3.py** or **ethers.js** on Pi for interaction. For privacy, use zk-SNARK proofs for sensitive commitments (handled off-chain).

- **Data storage:** Edge logs to a local **InfluxDB** or file, and optionally anchor aggregated results on IPFS or the blockchain.
- **Other Tools:** For P2P messaging, consider **libp2p** (IPFS router). For weather data, use a reliable API (e.g. NOAA or DarkSky) cached at a central server.

Step-by-Step Implementation Roadmap

1. **Hardware Setup:** Assemble a Pi5 with power metering sensors and test network connectivity. Install OS, Docker, Python environment. Build basic programs to read sensors and control relays. (*Proof of concept*)
2. **Local Forecasting:** Develop/port ML forecasting models (trained offline on historical data) to run on Pi. Test on sample data (verify [39†L63-L72]). Deploy as a Docker container on Pi.
3. **Smart Contracts:** Write and deploy identity and trading contracts on a testnet (or local Ganache). Implement order-book or auction logic. Test offer submission and trade matching. (*Use frameworks like Hardhat*)
4. **Agent Integration:** Code the Pi's EMS software: tie forecasting output to offer generation. Implement Web3 calls to publish offers to the contract. Use [48†L934-L942] as guide for TC logic.
5. **Safety Shield:** Implement local guard rails: ensure SoC limits and grid constraints are never violated. Hook this into the EMS decision loop.
6. **End-to-End Testing:** Run a few Pis (2–3) on a local network, each with a mock solar/load profile. Submit offers and verify smart contract matches and actuations. Debug latency and iterate.
7. **Domain Randomization:** Introduce forecast noise, simulate outages during testing. Ensure the system remains stable. Adjust trading frequency and timers as needed.
8. **Scaling & MLOps:** Set up a CI/CD pipeline to deploy software updates to all Pis. Configure automated data collection and retraining schedules.
9. **Security Hardening:** Move keys to HSM or secure enclave on Pi. Enforce TLS. Perform penetration testing on contract and network.
10. **Pilot Deployment:** Roll out to a small set of real prosumers. Monitor logs and performance (CPU, latency, transaction times). Tune accordingly.

Each stage should produce logs and a concise report. Testing should start in simulation, then hardware-in-the-loop, then live pilots.

Sources: Recent research and case studies on AI-powered P2P energy markets ¹ ⁸ ² ³ have been used to inform this design. These works illustrate exactly how **forecast-driven agentic decision-making** on edge devices integrates with **blockchain-based trade settlement** to form a secure, efficient decentralized grid. This answer synthesizes their insights into a concrete implementation plan.

¹ ⁵ ⁶ ⁸ EnergyFlow: Predictive trading platform for decentralized energy exchange - ScienceDirect
<https://www.sciencedirect.com/science/article/pii/S2352467725004564>

² ³ ⁴ ⁷ A Blockchain-Based Architecture for Energy Trading to Enhance Power Grid Stability
<https://www.mdpi.com/2079-9292/14/23/4629>